



Politecnico
di Torino



Exercises

Vision Transformers

Teaching Assistant:
Claudia Cuttano
claudia.cuttano@polito.it

Transformers: from images to sequences

High-level ViT pipeline:

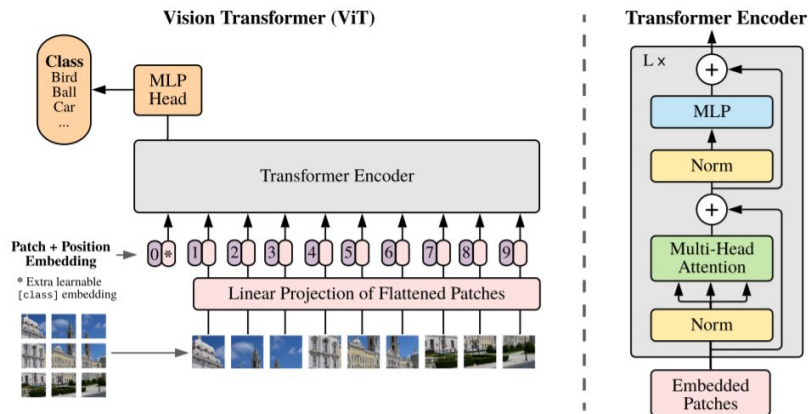
1. Convert image into a sequence of tokens
2. Add positional information
3. Process tokens with Transformer encoder blocks
4. Produce task-specific output (e.g. classification)

Transformers: from images to sequences

Transformers expect sequences: images are **grid**

- **Images:** $H \times W \times C$
- **Transformers:** $N \times D$

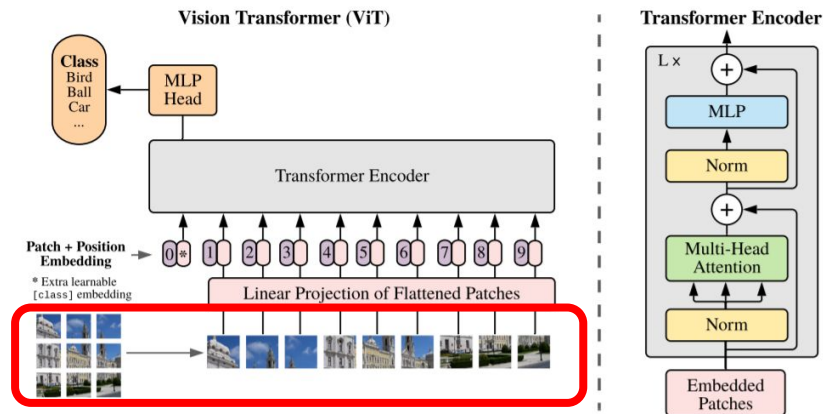
→ Split image into **patches** and treat them as tokens



Transformers: from images to sequences

Patchify the image:

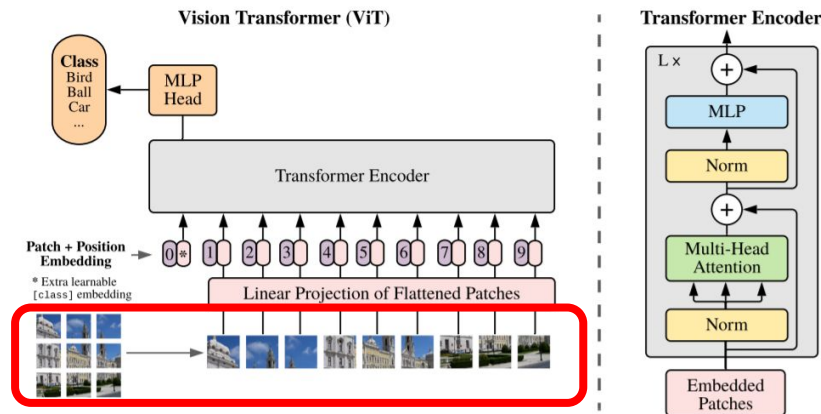
- Divide image into fixed-size patches (e.g. 16×16)
- Flatten each patch
- Linearly project to embedding dimension D



Transformers: from images to sequences

Why patches instead of pixels?

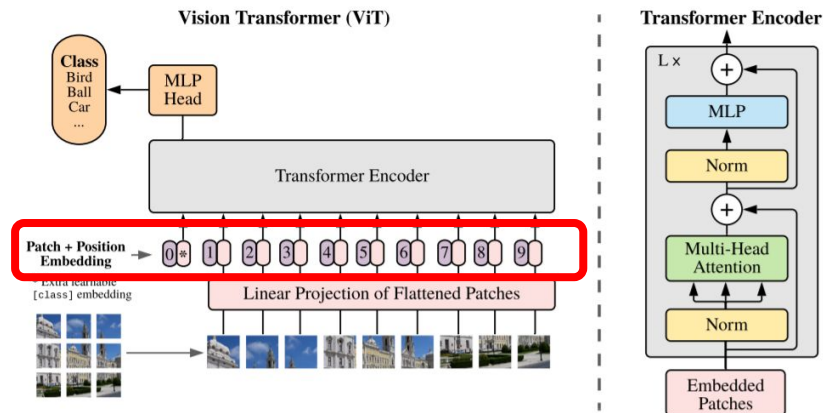
- Reduces sequence length
- Encodes local structure
- Makes attention computationally feasible



Transformers: from images to ordered sequences

Transformers are permutation-invariant

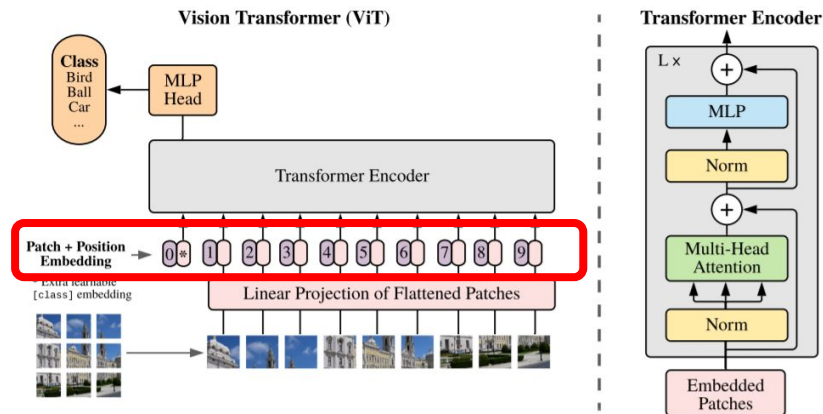
- Self-attention treats tokens as an unordered set
- Shuffling patch tokens does not change the output
- Images have strong spatial structure



Transformers: from images to ordered sequences

How do we encode spatial position?

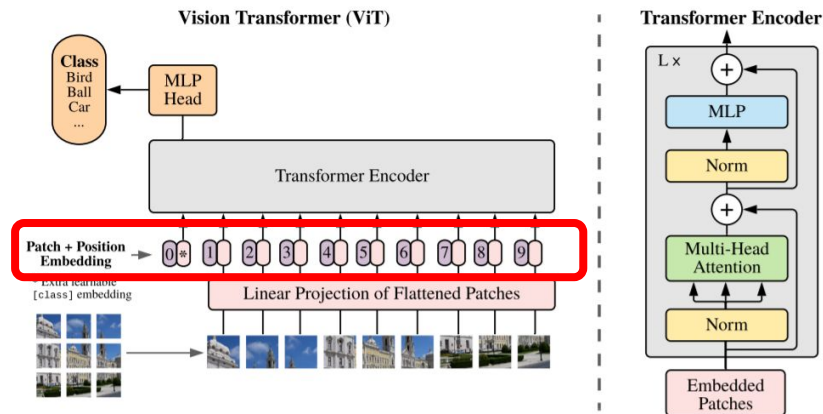
- Assign each patch a position-dependent embedding
- Add position embedding to patch embedding
- Enables attention to reason about where patches are



Transformers: from images to ordered sequences

Two common choices:

- Fixed **sinusoidal** positional embeddings → this is what you will implement!
- Learned positional embeddings

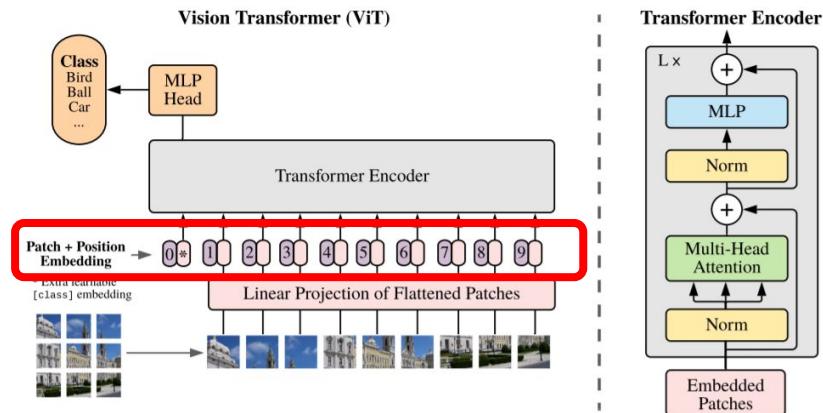


Transformers: from images to ordered sequences

Final token representation

For each patch: $\text{token} = \text{patch_embedding} + \text{positional_embedding}$

- Patch embedding $\rightarrow$ *what is in the patch*
- Positional embedding $\rightarrow$ *where the patch is*

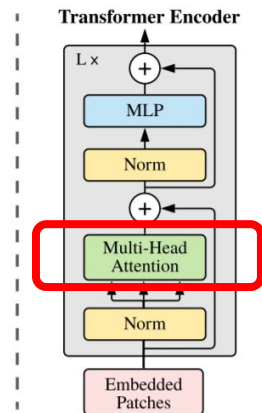
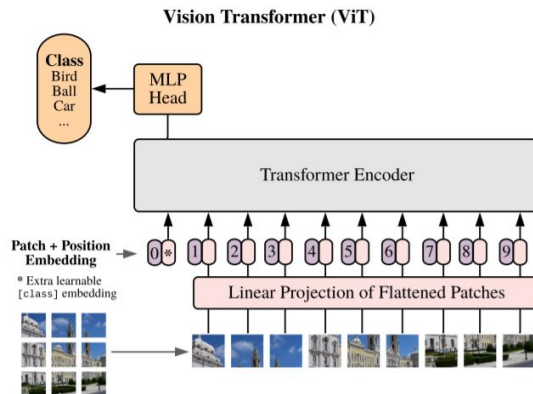


Transformers: Self-Attention

What happens after tokens are ready?

- We have a sequence of patch tokens
- Each token represents a local region
- Tokens must exchange information

**Self-attention lets
each patch look at
all others**

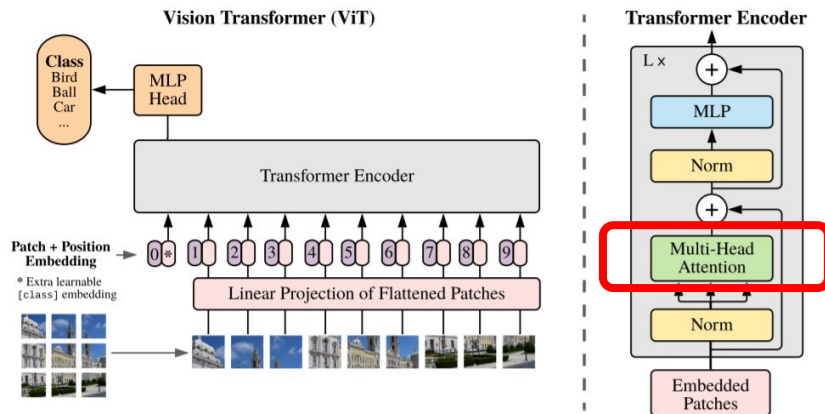


Transformers: Self-Attention

What is Self-Attention?

- Each token computes: *Which other tokens are relevant to me?*
- Output token = weighted combination of all tokens

Properties: *i)* global context and *ii)* data dependent interactions



Transformers: Self-Attention

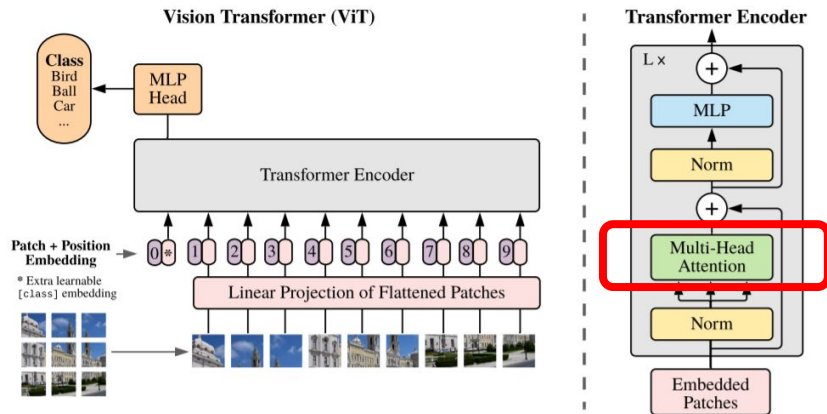
Query, Key, Value (QKV) - How attention is computed

Each token produces three vectors:

- **Query (Q)**: what I'm looking for
- **Key (K)**: what I contain
- **Value (V)**: what I provide

Attention scores:

- Similarity between Q and K
- Used to weight V



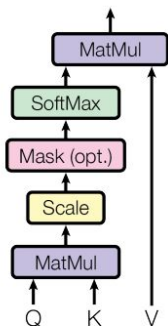
Transformers: Multi-Head Self-Attention

Single-Head Attention Is Limited: One attention = one view

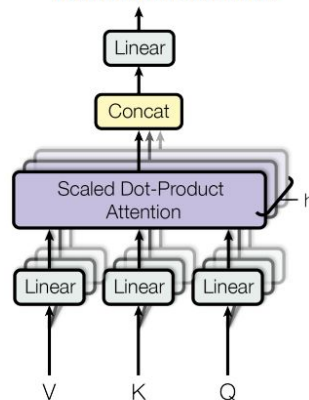
- Captures a single type of relation
- Might focus only on: Color, Shape, Spatial proximity

But images contain **multiple types of relationships!**

Scaled Dot-Product Attention



Multi-Head Attention



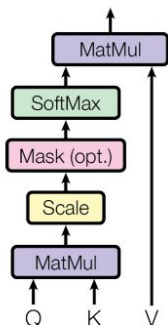
Transformers: Multi-Head Self-Attention

Multiple attention mechanisms in parallel

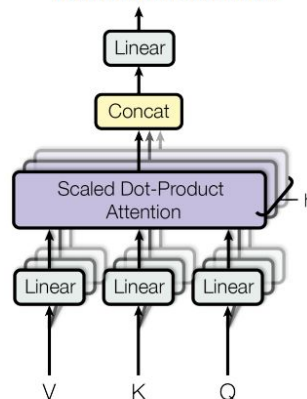
- Split embedding into multiple heads
- Each head: *i)* Own Q, K, V projection and *ii)* Own attention pattern

Outputs are concatenated and projected back.

Scaled Dot-Product Attention



Multi-Head Attention



Transformers: Multi-Head Self-Attention

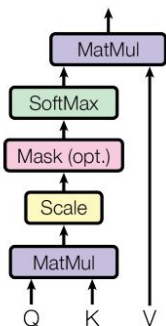
Benefits of multi-head attention

- Attend to different regions simultaneously
- Capture diverse relations
- Improves expressiveness without large cost

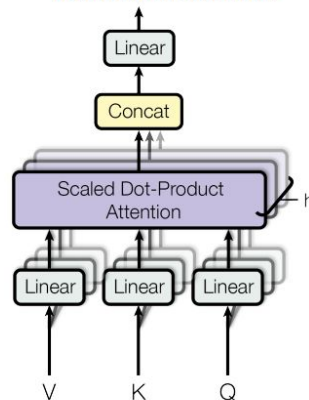
In practice:

- i) Linear projections: Q, K, V
- ii) Reshape into multiple heads
- iii) Apply attention per head
- iv) Concatenate head outputs
- v) Final linear projection

Scaled Dot-Product Attention



Multi-Head Attention



Lab Overview: Vision Transformers from Scratch

Step 1. Implement Core Attention Mechanics

- Scaled dot-product attention (Q, K, V)
- Test attention on toy examples

Step 2. Build Multi-Head Self-Attention

- Project inputs to Q/K/V
- Split into multiple heads
- Apply attention per head & merge outputs

Step 3. Construct a Transformer Encoder Block

- Multi-head self-attention
- Feed-forward MLP
- Residual connections + Layer Normalization



Step 4. Prepare Images for Transformers

- Split images into fixed-size patches
- Flatten and project patches into tokens

Step 5. Assemble a Vision Transformer

- Patch embedding layer
- Add classification token (CLS)
- Add positional embeddings
- Stack Transformer encoder blocks
- Classification head

Step 6. Train and Evaluate on CIFAR-10

- Train ViT from scratch
- Evaluate accuracy on validation/test set.

The template of the main code is available [here](#).

Now it's your turn, try!

